

Playground

bibinmuttappillil

March 2026

1 Introduction

2 Instantiation Example for MapV

```
theory MapExample
  imports Semantics VCExprHelper
begin
```

2.1 Type Definition

```
datatype 'p L = FunL "'p  $\Rightarrow$  'p L + 'p" ty ty
```

```
datatype 'a val0 = LitV0 lit | AbsV0 (the_absv: 'a)
type_synonym 'a val1 = "'a val0 L"
type_synonym 'a val10 = "'a val1 + 'a val0"
type_synonym 'a val2 = "'a val10 L"
type_synonym 'a val210 = "'a val2 + 'a val1 + 'a val0"
type_synonym 'a val3 = "'a val210 L"
type_synonym 'a val3210 = "'a val3 + 'a val210"
type_synonym 'a val321 = "'a val3 + 'a val2 + 'a val1"
type_synonym 'a valn = "('a, 'a val321) val"
```

2.2 Examples

```
value "IntV 2 :: unit valn"
```

```
abbreviation IntV0 where "IntV0 i  $\equiv$  LitV0 (LInt i)"
```

```
abbreviation TT where "TT  $\equiv$  TPrim TInt"
```

```
abbreviation m11 :: "'a val1" where "m11  $\equiv$  FunL (undefined(IntV0 3 := Inr (IntV0 2))) TT TT"
```

```
abbreviation m14 :: "'a valn" where "m14  $\equiv$  MapV (Inr (Inr m11))"
```

```
abbreviation m22 :: "'a val2" where "m22  $\equiv$  FunL (undefined(Inl m11 := Inr (Inr (IntV0 4)))) TT TT"
```

```
abbreviation m24 :: "'a valn" where "m24  $\equiv$  MapV (Inr (Inl m22))"
```

```
abbreviation m33 :: "'a val3" where "m33  $\equiv$  FunL (undefined(Inl m22 := Inr (Inr (Inr (IntV0 6))))) TT TT"
```

```
abbreviation m34 :: "'a valn" where "m34  $\equiv$  MapV (Inl m33)"
```

```
abbreviation mg3 :: "'a val3" where "mg3  $\equiv$  FunL (undefined(Inl m22 := Inr (Inr (Inl m11)))) (TMap TT TT) TT"
```

```
abbreviation mg4 :: "'a valn" where "mg4  $\equiv$  MapV (Inl mg3)"
```

```
abbreviation ms3 :: "'a val3" where "ms3  $\equiv$  FunL (undefined(Inr (Inr (IntV0 3)) := Inl m33)) TT (TMap TT (TPrim TInt))"
```

```
abbreviation ms4 :: "'a valn" where "ms4  $\equiv$  MapV (Inl ms3)"
```

2.3 Type Of Val

```

primrec tyL where "tyL (FunL f tk tv) = (tk, tv)"

fun ty321 :: "'a val321  $\Rightarrow$  ty  $\times$  ty" where
  "ty321 (Inr (Inr m)) = tyL m"
| "ty321 (Inr (Inl m)) = tyL m"
| "ty321 (Inl m) = tyL m"

instantiation L :: (type) mapval begin
  fun mapval_ty_L where "mapval_ty_L x = tyL x"
instance .. end

instantiation sum :: (mapval, mapval) mapval begin
  primrec mapval_ty_sum :: "'a + 'b  $\Rightarrow$  ty  $\times$  ty" where
    "mapval_ty_sum (Inl x) = mapval_ty x"
  | "mapval_ty_sum (Inr x) = mapval_ty x"
instance .. end

lemma mapval_ty_eq_ty321: "mapval_ty = ty321"
apply (rule, rename_tac x)
by (case_tac x rule: ty321.cases; simp)

fun key_ty where "key_ty (TMap tk _) = tk" | "key_ty _ = undefined"
fun val_ty where "val_ty (TMap _ tv) = tv" | "val_ty _ = undefined"

fun tmap_lvl :: "ty  $\Rightarrow$  nat" where
  "tmap_lvl (TMap tk tv) = max (1 + tmap_lvl tk) (tmap_lvl tv)"
| "tmap_lvl _ = 0"

primrec wf_L where
  "wf_L n (FunL _ tk tv) = (
    (tmap_lvl tk  $\leq$  n-1)  $\wedge$  (tmap_lvl tv  $\leq$  n)  $\wedge$ 
    ((tmap_lvl tk = n-1)  $\vee$  (tmap_lvl tv = n)))"

fun wf_ty :: "'a valn  $\Rightarrow$  bool" where
  "wf_ty (LitV v) = True"
| "wf_ty (AbsV v) = True"
| "wf_ty (MapV (Inr (Inr m))) = wf_L 1 m"
| "wf_ty (MapV (Inr (Inl m))) = wf_L 2 m"
| "wf_ty (MapV (Inl m)) = wf_L 3 m"

lemma map_level_gt_0: "tmap_lvl (TMap tv tk)  $\geq$  1" by auto

```

2.4 Select

```

fun toVal3210 :: "'a valn  $\Rightarrow$  'a val3210" where
  "toVal3210 (LitV v) = (Inr (Inr (Inr (LitV0 v))))"
| "toVal3210 (AbsV v) = (Inr (Inr (Inr (AbsV0 v))))"
| "toVal3210 (MapV (Inr (Inr m))) = (Inr (Inr (Inl m)))"

```

```

| "toVal3210 (MapV (Inr (Inl m))) = (Inr (Inl m))"
| "toVal3210 (MapV (Inl m)) = (Inl m)"

fun val3ToValn :: "'a val3210  $\Rightarrow$  'a valn" where
  "val3ToValn (Inr (Inr (Inr (LitV0 v)))) = (LitV v)"
| "val3ToValn (Inr (Inr (Inr (AbsV0 v)))) = (AbsV v)"
| "val3ToValn (Inr (Inr (Inl m))) = (MapV (Inr (Inr m)))"
| "val3ToValn (Inr (Inl m)) = (MapV (Inr (Inl m)))"
| "val3ToValn (Inl m) = (MapV (Inl m))"

fun selectImplAux :: "'a::absval val3210  $\Rightarrow$  'a val3210  $\Rightarrow$  'a val3210" where
  "selectImplAux (Inr (Inr (Inr (LitV0 v)))) _ = (Inr (Inr (Inr undefined)))"
| "selectImplAux (Inr (Inr (Inr (AbsV0 v)))) _ = (Inr (Inr (Inr undefined)))"
| "selectImplAux (Inr (Inr (Inl (FunL m _ _)))) (Inr (Inr (Inr k))) = Inr
(Inr (m k))"
| "selectImplAux (Inr (Inl (FunL m _ _))) (Inr (Inr k)) = Inr (m k)"
| "selectImplAux (Inl (FunL m _ _)) (Inr k) = (m k)"
| "selectImplAux m _ = toVal3210 (val_of_type (val_ty (type_of_val (val3ToValn
m))))"

fun selectImpl :: "'a::absval valn  $\Rightarrow$  'a valn  $\Rightarrow$  'a valn" where
  "selectImpl m k = val3ToValn (selectImplAux (toVal3210 m) (toVal3210 k))"

lemma "selectImpl mg4 m24 = m14" by simp

```

2.5 Helper Case Distinction

```

thm val3ToValn.cases
lemma ValnCases:
  "( $\bigwedge v. x = \text{LitV } v \implies P$ )  $\implies$ 
( $\bigwedge v. x = \text{AbsV } v \implies P$ )  $\implies$ 
( $\bigwedge f \text{ tk tv. } x = \text{MapV (Inr (Inr (FunL f tk tv)))} \implies P$ )  $\implies$ 
( $\bigwedge f \text{ tk tv. } x = \text{MapV (Inr (Inl (FunL f tk tv)))} \implies P$ )  $\implies$ 
( $\bigwedge f \text{ tk tv. } x = \text{MapV (Inl (FunL f tk tv))} \implies P$ )  $\implies P$ "
  by (metis L.exhaust sum.collapse val.exhaust)

```

2.6 Helper Injectivity Lemmas for toVal3210 and val3ToValn

```

lemma valBij: "toVal3210 (val3ToValn x) = x"
  by (cases x rule: val3ToValn.cases; simp)

lemma valBij2: "val3ToValn (toVal3210 x) = x"
  by (cases x rule: toVal3210.cases; simp)

lemma toVal3210_inj:
  assumes "toVal3210 x = toVal3210 y"
  shows "x = y"
  apply (cases x rule: toVal3210.cases; cases y rule: toVal3210.cases)
  using assms by auto

```

2.7 Store

```

fun storeImplAux :: "'a val3210 ⇒ 'a val3210 ⇒ 'a val3210 ⇒ 'a val3210"
where
  "storeImplAux (Inr (Inr (Inl (FunL m tk tv)))) (Inr (Inr (Inr k))) (Inr
(Inr v))
    = (Inr (Inr (Inl (FunL (m(k := v)) tk tv))))"
  | "storeImplAux (Inr (Inl (FunL m tk tv))) (Inr (Inr k)) (Inr v)
    = (Inr (Inl (FunL (m(k := v)) tk tv)))"
  | "storeImplAux (Inl (FunL m tk tv)) (Inr k) v
    = (Inl (FunL (m(k := v)) tk tv))"
  | "storeImplAux x _ _ = x"

fun storeImpl :: "'a::absval valn ⇒ 'a valn ⇒ 'a valn ⇒ 'a valn" where
  "storeImpl m k v = (if type_of_val m = TMap (type_of_val k) (type_of_val
v)
    then val3ToValn (storeImplAux (toVal3210 m) (toVal3210 k) (toVal3210 v))
    else m)"

```

```

lemma "type_of_val (LitV (LInt 42)) = TT" by simp
lemma "val_ty (type_of_val mg4) = TT" by simp
lemma "selectImpl (storeImpl mg4 m24 (LitV (LInt 42))) m24
    = (LitV (LInt 42))" by simp

```

2.8 Well Formedness

```

inductive wf where
  wfLitV: "wf (LitV v)" | wfAbsV: "wf (AbsV v)" |
  wfMapV: "[[ wf_ty m; (∀k. wf (selectImpl m k));
    (∀k. (¬wf k ∨ type_of_val k ≠ key_ty (type_of_val m)) → (selectImpl
m k) = val_of_type (val_ty (type_of_val m)));
    (∀k. type_of_val (selectImpl m k) = val_ty (type_of_val m))
  ] ⇒ wf m"

```

```

fun valid_mapty :: "ty ⇒ bool" where "valid_mapty t = (tmap_lvl t ≤ 3)"

```

2.8.1 Bijection between tmaplvl and sum type levels

```

lemma C0Inrrr:
  assumes "tmap_lvl (type_of_val v) = 0"
  shows "∃v'. toVal3210 v = Inr (Inr (Inr v'))"
  apply (cases v)
  apply auto
  by (metis assms map_level_gt_0 not_one_le_zero
    type_of_val.simps(3))

lemma InrrrC0:
  assumes "toVal3210 v = Inr (Inr (Inr v'))"

```

```

shows "tmap_lvl (type_of_val v) = 0"
using assms tmap_lvl.simps(4) toVal3210.elims by force

lemma Inr1C1:
  assumes "wf_ty v"
  assumes "toVal3210 v = Inr (Inr (Inl v'))"
  shows "tmap_lvl (type_of_val v) = 1"
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (cases v') using assms
  by fastforce+

lemma C1Inr1:
  assumes "wf_ty v"
  assumes "tmap_lvl (type_of_val v) = 1"
  shows " $\exists v'. \text{toVal3210 } v = \text{Inr (Inr (Inl v'))}$ "
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (case_tac m) using assms apply auto
  apply (case_tac m) using assms apply auto
done

lemma Inr1C2:
  assumes "wf_ty v"
  assumes "toVal3210 v = Inr (Inl v')"
  shows "tmap_lvl (type_of_val v) = 2"
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (cases v') using assms
  by fastforce+

lemma C2Inr1:
  assumes "wf_ty v"
  assumes "tmap_lvl (type_of_val v) = 2"
  shows " $\exists v'. \text{toVal3210 } v = \text{Inr (Inl v')}$ "
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (case_tac m) using assms apply auto
  apply (case_tac m) using assms apply auto
done

lemma InlC3:
  assumes "wf_ty v"
  assumes "toVal3210 v = (Inl v')"
  shows "tmap_lvl (type_of_val v) = 3"
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (cases v') using assms
  by fastforce+

```

```

lemma C3Inl:
  assumes "wf_ty v"
  assumes "tmap_lvl (type_of_val v) = 3"
  shows "∃ v'. toVal3210 v = (Inl v')"
  apply (cases v rule: toVal3210.cases)
  using assms apply auto
  apply (case_tac m) using assms apply auto
  apply (case_tac m) using assms apply auto
done

```

2.8.2 Proving well formedness of a simple map

```

fun toVal0 :: "'a valn ⇒ 'a val0" where "toVal0 (LitV l) = LitV0 l" | "toVal0
_ = undefined"
fun fAdd1 where "fAdd1 (IntV0 x) = Inr (IntV0 (x+1))" | "fAdd1 _ = Inr (toVal0
(val_of_type TT))"
abbreviation mAdd1 :: "'a::absval val1" where "mAdd1 ≡ FunL fAdd1 (TPrim
TInt) (TPrim TInt)"
abbreviation vAdd1 :: "'a::absval valn" where "vAdd1 ≡ MapV (Inr (Inr mAdd1))"

lemma wfvotTT: "(type_of_val ((val_of_type TT)::'a::absval valn) = TT ∧ wf
((val_of_type TT)::'a::absval valn))"
  by (metis (mono_tags, lifting) someI_ex tint_intv type_of_lit.simps(2) type_of_val.simps(1)
    val_of_type.simps wflitV)

lemma mAdd1Typesafe:
  shows "type_of_val (selectImpl vAdd1 k) = (TPrim TInt)"
  apply (cases k rule: toVal3210.cases)
  apply (metis (no_types, lifting) fAdd1.elims int_inverse_3 selectImpl.simps
    selectImplAux.simps(3)
      toVal0.simps(1) toVal3210.simps(1,3) type_of_lit.simps(2) type_of_val.simps(1)
      val3ToValn.simps(1)
        wfvotTT)
  using wfvotTT tint_intv
  apply (metis (no_types, opaque_lifting) fAdd1.simps(4) selectImpl.simps
    selectImplAux.simps(3)
      toVal0.simps(1) toVal3210.simps(1,3) val3ToValn.simps(2) valBij valBij2)
  apply (metis mapval_ty_eq_ty321 selectImpl.simps selectImplAux.simps(11)
    snd_conv toVal3210.simps(3)
      ty321.simps(1) tyL.simps type_of_val.simps(3) valBij2 val_ty.simps(1)
      wfvotTT)
  apply (metis mapval_ty_eq_ty321 selectImpl.simps selectImplAux.simps(10)
    snd_conv toVal3210.simps(3,4)
      ty321.simps(1) tyL.simps type_of_val.simps(3) valBij2 val_ty.simps(1)
      wfvotTT)
  apply (metis mapval_ty_eq_ty321 selectImpl.simps selectImplAux.simps(09)
    snd_conv toVal3210.simps(3,5)
      ty321.simps(1) tyL.simps type_of_val.simps(3) valBij2 val_ty.simps(1)
      wfvotTT)
done

```

```

lemma votTTpreserved: "val3ToValn (Inr (Inr (Inr (toVal0 (val_of_type (TT)))))
= (val_of_type (TT))"
  by (metis int_inverse_3 toVal0.simps(1) val3ToValn.simps(1) wfvotTT)

```

```

lemma vAdd1default: "(¬wf k ∨ type_of_val k ≠ key_ty (type_of_val vAdd1))
⇒ (selectImpl vAdd1 k) = val_of_type (val_ty (type_of_val vAdd1))"
  apply (cases "k" rule: toVal3210.cases; cases "type_of_val k"; simp)
  apply (rename_tac v t, case_tac v; simp) using votTTpreserved apply
fastforce
  using wflitV wfAbsV toVal3210_inj valBij apply blast
  using votTTpreserved apply auto[1]
  using votTTpreserved apply auto[1]
  using wflitV wfAbsV toVal3210_inj valBij apply blast+
done

```

```

lemma vAdd1wfSelect: "wf (selectImpl vAdd1 k)"
  using mAdd1Typesafe tint_intv wf.simps by blast

```

```

lemma wf_vAdd1: "wf vAdd1"
  using vAdd1wfSelect vAdd1default mAdd1Typesafe wfMapV[of vAdd1] by auto

```

2.8.3 Well formdness of a higher order map

```

locale valoftype =
  assumes VOT: "∧t. valid_mapty t ⇒ closed t ⇒ (type_of_val ((val_of_type
t)::'a::absval valn) = t ∧ wf ((val_of_type t)::'a::absval valn))"
begin

fun toVal1 :: "'a valn ⇒ 'a val1" where "toVal1 (MapV (Inr (Inr m))) = m"
| "toVal1 _ = undefined"
fun val00f10 :: "'a val10 ⇒ 'a val0" where "val00f10 (Inr v0) = v0" | "val00f10
_ = undefined"
abbreviation TMII where "TMII ≡ TMap (TPrim TInt) (TPrim TInt)"
fun compint where
  "compint g f (IntV0 x) = (g (val00f10 (f (IntV0 x))))"
| "compint g f _ = Inr (toVal0 (val_of_type (TT)))"
fun hof where
  "hof (Inl (FunL f tk tv)) =
    (if (tk, tv) = (TT, TT) ∧ wf (MapV (Inr (Inr (FunL f tk tv))))
    then Inr (Inl (FunL (compint fAdd1 f) TT TT))
    else Inr (Inl (toVal1 (val_of_type (TMII)))))"
| "hof _ = Inr (Inl (toVal1 (val_of_type (TMII)))))"
abbreviation hom :: "'a::absval val2" where "hom ≡ FunL hof TMII TMII"
abbreviation homV :: "'a::absval valn" where "homV ≡ MapV (Inr (Inl hom))"

```

```

lemma "wf_ty homV" by auto

```

```

lemma "(val_ty (type_of_val homV) = TMII)" by simp

```



```

lemma "(key_ty (type_of_val homV) = TMII)" by simp

lemma "type_of_val k = TMII  $\longrightarrow$  ( $\exists k'. k = \text{MapV } k'$ )" apply (cases k) by auto

lemma kTMII:
  assumes "wf_ty k"
  assumes "type_of_val k = TMII"
  shows " $\exists f. k = \text{MapV } (\text{Inr } (\text{Inr } (\text{FunL } f \text{ TT TT})))$ "
proof -
  have "tmap_lvl (type_of_val k) = 1" using assms by simp
  then obtain k' where "toVal3210 k = Inr (Inr (Inl k'))" using C1Inrr1 assms
  by blast
  then have K: "k = MapV (Inr (Inr k'))" using toVal3210.elims by auto
  then have "wf_L 1 k'" using assms by force
  then show ?thesis
    apply (cases k')
    using K assms(2) by auto
qed

lemma homvotdef: "tyL (toVal1 (val_of_type (TMap TT TT))) = (TT, TT)" using
  VOT
  by (smt (verit) One_nat_def Suc_eq_plus1 closed.simps(2,4) tmap_lvl.simps(1,3)
  kTMII
  le_eq_less_or_eq max_0_1(2) not_less_eq_eq numeral_Bit1 numeral_One plus_1_eq_Suc
  toVal1.simps(1)
  tyL.simps val.distinct(3) valid_mapty.elims(3) wf.simps wf_ty.elims(1)
  wf_ty.simps(2)
  zero_less_two)

lemma votTMIIpreserved: "val3ToValn (Inr (Inr (Inl (toVal1 (val_of_type (TMII))))))
= (val_of_type (TMII))"
  by (metis (no_types, lifting) One_nat_def Suc_eq_plus1 closed.simps(2,4)
  tmap_lvl.simps(1,3) kTMII le_eq_less_or_eq max_0_1(2) not_less_eq_eq
  numeral_Bit1
  numeral_One plus_1_eq_Suc toVal1.simps(1) val3ToValn.simps(3) valid_mapty.elims(3)
  wf.simps
  wf_ty.simps(1,2) zero_less_two VOT)

lemma wff: "type_of_val (selectImpl homV k) = TMII"
  apply (case_tac k rule: ValnCases; (simp add: valBij2))
  using homvotdef VOT by auto

lemma validClosedTMII: "valid_mapty (TMap TT TT)  $\wedge$  closed (TMap TT TT)" by
  simp

lemma compwf:
  assumes "a = (MapV (Inr (Inr (FunL f TT TT))))"

```

```

assumes "b = (MapV (Inr (Inr (FunL (compint fAdd1 f) TT TT))))"
assumes "wf a"
shows "wf b"
proof -
  have wft: "wf_ty b" by (simp add: assms(2))

  have wf1: "∀k. wf (selectImpl b k)"
  apply rule
  apply (case_tac k rule: toVal3210.cases; (simp add: assms))
  apply (rename_tac v, case_tac v; (simp add: assms))
  apply (metis val_of_type.simps votTTPreserved wfvotTT)
  apply (metis fAdd1.elims val3ToValn.simps(1) votTTPreserved wflitV wfvotTT)
  apply (metis val_of_type.simps votTTPreserved wfvotTT)
  apply (metis toVal3210.simps(1) val0.exhaust val3ToValn.simps(2) valBij2
wfAbsV
  wflitV)
  apply (metis valBij2 val_of_type.simps wfvotTT)+
  done
  have KTT: "key_ty (type_of_val b) = TT" by (simp add: assms(2))
  have wf2: "∀k. (¬wf k ∨ type_of_val k ≠ key_ty (type_of_val b)) → (selectImpl
b k) = val_of_type (val_ty (type_of_val b))"
  proof (rule) fix k
    show "(¬wf k ∨ type_of_val k ≠ key_ty (type_of_val b)) → (selectImpl
b k) = val_of_type (val_ty (type_of_val b))"
    proof (cases k rule: toVal3210.cases)
      case (1 v)
      then show ?thesis apply (cases v)
      using assms(2) votTTPreserved apply auto[1]
      apply (simp add: KTT wflitV)
      using assms(2) votTTPreserved apply auto
      done
    next
      case (2 v)
      then show ?thesis
      using assms(2) votTTPreserved by auto
    qed (auto simp add: assms valBij2)
  qed

  have wf3: "∀k. type_of_val (selectImpl b k) = val_ty (type_of_val b)"
  apply rule
  apply (case_tac k rule: toVal3210.cases; (simp add: assms))
  apply (rename_tac v, case_tac v; simp)
  apply (metis val_of_type.simps votTTPreserved wfvotTT)
  apply (metis (no_types, lifting) fAdd1.elims type_of_lit.simps(2) type_of_val.simps(1)
val3ToValn.simps(1) votTTPreserved wfvotTT)
  apply (metis val_of_type.simps votTTPreserved wfvotTT)
  apply (metis val_of_type.simps votTTPreserved wfvotTT)
  apply (metis valBij2 val_of_type.simps wfvotTT)+
  done

```

```

    have wf2: "∀k. (¬wf k ∨ type_of_val k ≠ key_ty (type_of_val b)) → (selectImpl
b k) = val_of_type (val_ty (type_of_val b))"
    apply rule
    apply (case_tac k rule: toVal3210.cases; (simp add: assms valBij2))
    apply (rename_tac v, case_tac v; simp)
    using votTTPreserved apply fastforce
    using wfLitV apply blast
    using votTTPreserved apply fastforce+
    done

    show ?thesis using wft wf1 wf2 wf3 wfMapV by simp
qed

```

```

lemma vhomVwfSelect: "wf (selectImpl homV k)"
  apply (cases k rule: toVal3210.cases; simp)
  using VOT votTMIIpreserved apply auto[1]
  using VOT votTMIIpreserved apply auto[1]
  apply (rename_tac m, case_tac m)
  using VOT validClosedTMII val_of_type.simps
  apply (simp add: valBij2)
  using valoftype.compwf valoftype.intro votTMIIpreserved apply fastforce
  apply (metis VOT valBij2 val_of_type.simps validClosedTMII)
  apply (metis VOT valBij2 val_of_type.simps validClosedTMII)
  done

```

```

lemma homVdef:
  "(∀k. (¬wf k ∨ type_of_val k ≠ key_ty (type_of_val homV)) → (selectImpl
homV k) = val_of_type (val_ty (type_of_val homV)))"
proof rule
  fix k
  show "(¬wf k ∨ type_of_val k ≠ key_ty (type_of_val homV)) → (selectImpl
homV k) = val_of_type (val_ty (type_of_val homV))"
    apply simp
    apply (cases k rule: ValnCases)
    apply (simp add: valBij2)
    using votTMIIpreserved apply auto[1]
    apply (simp add: votTMIIpreserved)
    using votTMIIpreserved apply force
    using votTMIIpreserved apply force
    apply (simp add: valBij2)+
    done
qed

```

```

lemma "wf homV" using wff vhomVwfSelect wf.simps[of homV] homVdef by auto

end

```

2.8.4 Some more general wf properties

lemma wf_impl_wf_ty: "wf k $\implies$ wf_ty k" using wf.cases by force

```

lemma
  assumes "wf (MapV (Inl (FunL f tk tv)))"
  assumes "wf k"
  assumes "type_of_val k = key_ty (type_of_val (MapV (Inl (FunL f tk tv))))"
  shows " $\exists k'$ . toVal3210 k = Inr (k')"
proof -
  have "tmap_lvl (type_of_val (MapV (Inl (FunL f tk tv)))) = 3"
    using InlC3 wf.simps assms(1) toVal3210.simps wf_impl_wf_ty by force
  then have "tmap_lvl (key_ty (type_of_val (MapV (Inl (FunL f tk tv))))  $\leq$  2"
  by auto
  then have "tmap_lvl (type_of_val k)  $\leq$  2" using assms by auto
  then show ?thesis using assms(2) wf_impl_wf_ty
    by (metis InlC3 One_nat_def Suc_1 Suc_n_not_le_n numeral_3_eq_3 old.sum.exhaust)
qed

```

2.9 Array Axiom Update

Property to prove: $(m[k] := v)[k] == v$ or select (store m k v) k = v

```

lemma ArrayAxUpdate:
  assumes "wf M"
  assumes "wf k"
  assumes "wf v"
  assumes "type_of_val M = TMap (type_of_val k) (type_of_val v)"
  shows "selectImpl (storeImpl M k v) k = v"
proof (cases M rule: ValnCases)
  case (1 v)
  then show ?thesis using assms by force
next
  case (2 v)
  then show ?thesis using assms by force
next
  case (3 f tk tv)
  then show ?thesis proof (cases "(tmap_lvl tv  $\leq$  0)")
    case True
    then have C: "tmap_lvl (type_of_val k) = 0  $\wedge$  tmap_lvl (type_of_val v)  $\leq$  0"
    using assms wf_impl_wf_ty "3" by fastforce
    obtain k' where K: "toVal3210 k = Inr (Inr (Inr k'))"
    using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
    by (cases k; simp)
    obtain v' where V: "toVal3210 v = Inr (Inr (Inr v'))"
    using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
    by (cases v; simp)
    show ?thesis using assms wf_impl_wf_ty K V "3" by (simp add: toVal3210_inj

```

```

valBij)
  next
    case False
    then have C: "tmap_lvl (type_of_val k) ≤ 0 ∧ tmap_lvl (type_of_val
v) = 1"
      using assms wf_impl_wf_ty "3" by fastforce
    obtain k' where K: "toVal3210 k = Inr (Inr (Inr k'))"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      by (cases k; simp)
    obtain v' where V: "toVal3210 v = Inr (Inr (Inl v'))"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases v; simp) by fastforce
    show ?thesis using assms wf_impl_wf_ty K V "3" by (simp add: toVal3210_inj
valBij)
  qed
next
  case (4 f tk tv)
  then show ?thesis proof (cases "(tmap_lvl tv ≤ 1)")
    case True
    then have C: "tmap_lvl (type_of_val k) = 1 ∧ tmap_lvl (type_of_val
v) ≤ 1"
      using assms wf_impl_wf_ty "4" by fastforce
    obtain k' where K: "toVal3210 k = Inr (Inr (Inl k'))"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases k; simp) by fastforce
    obtain v' where V: "toVal3210 v = Inr (Inr v')"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases v; simp) by fastforce
    show ?thesis using assms wf_impl_wf_ty K V "4" by (simp add: toVal3210_inj
valBij)
  next
    case False
    then have C: "tmap_lvl (type_of_val k) ≤ 1 ∧ tmap_lvl (type_of_val
v) = 2"
      using assms wf_impl_wf_ty "4" by fastforce
    obtain k' where K: "toVal3210 k = Inr (Inr k')"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases k; simp) by fastforce
    obtain v' where V: "toVal3210 v = Inr (Inl v')"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases v; simp) by fastforce
    show ?thesis using assms wf_impl_wf_ty K V "4" by (simp add: toVal3210_inj
valBij)
  qed
next
  case (5 f tk tv)
  then show ?thesis proof (cases "(tmap_lvl tv ≤ 2)")
    case True
    then have C: "tmap_lvl (type_of_val k) = 2 ∧ tmap_lvl (type_of_val
v) ≤ 2"

```

```

      using assms wf_impl_wf_ty "5" by fastforce
    obtain k' where K: "toVal3210 k = Inr (Inl k')"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases k; simp) by fastforce
    obtain v' where V: "toVal3210 v = Inr v'"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases v; simp) by fastforce
    show ?thesis using assms wf_impl_wf_ty K V "5" by (simp add: toVal3210_inj
valBij)
  next
    case False
    then have C: "tmap_lvl (type_of_val k) ≤ 2 ∧ tmap_lvl (type_of_val
v) = 3"
      using assms wf_impl_wf_ty "5" by fastforce
    obtain k' where K: "toVal3210 k = Inr k'"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases k; simp) by fastforce
    obtain v' where V: "toVal3210 v = Inl v'"
      using assms wf_impl_wf_ty C3Inl C2Inrl C1Inrrl C0Inrrr C
      apply (cases v; simp) by fastforce
    show ?thesis using assms wf_impl_wf_ty K V "5" by (simp add: toVal3210_inj
valBij)
  qed
qed

```

2.10 Array Axiom Stable

lemma ArrayAxStable:

```

  assumes "x ≠ y"
  shows "selectImpl (storeImpl M x v) y = selectImpl M y"
  apply (cases "(toVal3210 M, toVal3210 x, toVal3210 v)" rule: storeImplAux.cases;
(simp add: assms);
    cases y rule: toVal3210.cases; (simp add: valBij); auto)
  apply (metis assms toVal3210.simps toVal3210_inj)+
  done

```

2.11 Array Axiom Extensionality

2.11.1 Extensionality

```

lemma extensionalityAux:
  assumes "wf (MapV m)"
  assumes "wf (MapV n)"
  assumes "selectImplAux (toVal3210 (MapV m)) = selectImplAux (toVal3210 (MapV
n))"
  assumes "type_of_val (MapV m) = type_of_val (MapV n)"
  shows "m = n"
  proof (cases m rule: ty321.cases)
    case (1 m')

```

```

then show ?thesis
proof -
  have "tmap_lvl (type_of_val (MapV m)) = 1"
    using "1" InrrlC1 assms(1) toVal3210.simps(3) wf_impl_wf_ty by fastforce
  then have "tmap_lvl (type_of_val (MapV n)) = 1"
    using assms(4) by simp
  then obtain n' where "n = Inr (Inr n')"
    using C1Inrrl assms(2) toVal3210_inj val.inject(3) val3ToValn.simps(3)
valBij
    wf_impl_wf_ty by metis
  then show ?thesis
  proof (cases m')
    case (FunL m'' tmk tmv)
    then show ?thesis
    proof (cases n')
      case (FunL n'' tnk tnv)
      have "(tmk, tmv) = (tnk, tnv)" using assms(4) 1 FunL <m' = FunL
m'' tmk tmv>
        by (metis <n = Inr (Inr n')> prod.collapse ty.inject(4) ty321.simps(1)
tyL.simps(1)
          type_of_val.simps(3) mapval_ty_eq_ty321)
      moreover have "m'' = n'"
      proof (rule ext)
        fix k show "m'' k = n'' k"
        using assms(3) 1 FunL <m' = FunL m'' tmk tmv> <n' = FunL n'' tnk
tnv>
          selectImplAux.simps(3) sum.inject(2) toVal3210.simps(3)
          by (metis <n = Inr (Inr n')>))
      qed
      ultimately show ?thesis using 1 FunL <m' = FunL m'' tmk tmv> <n'
= FunL n'' tnk tnv>
        using <n = Inr (Inr n')> by force
      qed
    qed
  qed
next
case (2 m')
then show ?thesis
proof -
  have "tmap_lvl (type_of_val (MapV m)) = 2"
    using "2" InrlC2 assms(1) toVal3210.simps(4) wf_ty.simps(4) wf_impl_wf_ty
    by fastforce
  then have "tmap_lvl (type_of_val (MapV n)) = 2"
    using assms(4) by simp
  then obtain n' where N: "n = Inr (Inl n')"
    using C2Inrl assms(2) toVal3210_inj val.inject(3) val3ToValn.simps(4)
valBij
    by (metis wf_impl_wf_ty)
  then show ?thesis
    using assms(3,4) 2 N apply (cases m'; cases n')

```

```

      by (auto simp: fun_eq_iff dest: spec[of _ "Inr (Inr _)"])
    qed
  next
    case (3 m')
    then show ?thesis
    proof -
      have "tmap_lvl (type_of_val (MapV m)) = 3"
      using "3" InlC3 assms(1) toVal3210.simps(5) wf_ty.simps(5) wf_impl_wf_ty
    by fastforce
      then have "tmap_lvl (type_of_val (MapV n)) = 3"
      using assms(4) by simp
      then obtain n' where N: "n = Inl n'"
      using C3Inl assms toVal3210_inj val.inject(3) val3ToValn.simps(5) valBij
      by (metis wf_impl_wf_ty)
      then show ?thesis
      using assms(3,4) 3 N apply (cases m'; cases n')
      by (auto simp: fun_eq_iff dest: spec[of _ "Inr _"])
    qed
  qed

lemma extensionalityMapVWeak:
  assumes "wf (MapV m)"
  assumes "wf (MapV n)"
  assumes "type_of_val (MapV m) = type_of_val (MapV n)"
  assumes "selectImpl (MapV m) = selectImpl (MapV n)"
  shows "m = n"
  by (metis (no_types, lifting) ext extensionalityAux assms(1,2,3,4) selectImpl.simps
      valBij)

lemma extensionalityMapV:
  assumes "wf (MapV m)"
  assumes "wf (MapV n)"
  assumes "type_of_val (MapV m) = type_of_val (MapV n)"
  assumes "∀k. (wf k ∧ type_of_val k = key_ty (type_of_val (MapV m))) →
    selectImpl (MapV m) k = selectImpl (MapV n) k"
  shows "m = n"
  proof -
    have "∀k. selectImpl (MapV m) k = selectImpl (MapV n) k"
    proof -
      fix k show "selectImpl (MapV m) k = selectImpl (MapV n) k"
      proof (cases "(wf k ∧ type_of_val k = key_ty (type_of_val (MapV m)))")
        case True
        then show ?thesis using assms(4) by force
      next
        case False
        then have PM: "(¬wf k ∨ type_of_val k ≠ key_ty (type_of_val (MapV
m)))" by simp
        then have M: "(selectImpl (MapV m) k) = val_of_type (val_ty (type_of_val

```



```

(MapV m)))"
  using assms(1) wf.cases by fastforce
  have "(¬wf k ∨ type_of_val k ≠ key_ty (type_of_val (MapV n)))"
    using assms PM by simp
  then have N: "(selectImpl (MapV n) k) = val_of_type (val_ty (type_of_val
(MapV n)))"
    using assms(2) wf.cases by fastforce
  then show ?thesis using M N assms by auto
qed
qed
then show ?thesis using extensionalityMapVWeak using assms(1,2,3) by blast
qed

```

2.12 Select and Store is closed under wf

Lemma for return value of invalid select

```

lemma wf_undefined: "(wf (val3ToValn (Inr (Inr (Inr undefined)))))"
  by (metis wfAbsV wfLitV val0.exhaust val3ToValn.simps(1,2))

```

```

lemma selectClosedWf:
  assumes "wf m"

  shows "wf (selectImpl m k)"
  by (metis wf.cases assms(1) selectImpl.elims selectImplAux.simps(1,2) toVal3210.simps(1,2)
wf_undefined)

```

```

lemma storePreserveTy:
  shows "type_of_val m = type_of_val (storeImpl m k v)"
  by (cases m rule: ValnCases;
    (simp);
    cases "((toVal3210 m), (toVal3210 k), (toVal3210 v))" rule: storeImplAux.cases;
    (simp))

```

```

lemma storeClosedWf3:
  assumes "wf m" "wf k" "wf v"
  assumes "type_of_val m = TMap (type_of_val k) (type_of_val v)"
  shows "type_of_val (selectImpl (storeImpl m k v) x) = val_ty (type_of_val
(storeImpl m k v))"
  using assms
  apply (cases "x = k"; simp)
  apply (metis ArrayAxUpdate selectImpl.simps storeImpl.simps storePreserveTy
val_ty.simps(1))
  by (metis (no_types, opaque_lifting) ArrayAxStable selectImpl.elims storeImpl.simps
storePreserveTy
ty.simps(14,16) type_of_val.simps(1,2) wf.cases)

```

```

lemma storeClosedWf2:
  assumes "wf m" "wf k" "wf v"
  assumes "type_of_val m = TMap (type_of_val k) (type_of_val v)"

```

```

shows "(∀ k'. wf (selectImpl (storeImpl m k v) k'))"
by (metis ArrayAxStable ArrayAxUpdate assms(1,2,3,4) selectClosedWf)

lemma storeClosedWf1:
  assumes "wf m" "wf k" "wf v"
  assumes "type_of_val m = TMap (type_of_val k) (type_of_val v)"
  shows "wf_ty (storeImpl m k v)"
  using assms wf_impl_wf_ty apply simp

  by (cases m rule: ValnCases; simp;
      cases "(toVal3210 k)" rule: val3ToValn.cases; simp;
      cases "(toVal3210 v)" rule: val3ToValn.cases; fastforce)

lemma storeClosedWfDef:
  assumes "wf m" "wf k'" "wf v"
  assumes "type_of_val m = TMap (type_of_val k') (type_of_val v)"
  shows "(∀ k. (¬ wf k ∨ type_of_val k ≠ key_ty (type_of_val (storeImpl m
k' v))) → (selectImpl (storeImpl m k' v) k) = val_of_type (val_ty (type_of_val
(storeImpl m k' v))))"
  by (metis (no_types, opaque_lifting) ArrayAxStable assms(1,2,4) key_ty.simps(1)
      storePreserveTy ty.distinct(9) ty.simps(16) type_of_val.simps(1,2)
      wf.cases)

lemma storeClosedWf:
  assumes "wf m"
  assumes "wf k"
  assumes "wf v"
  shows "wf (storeImpl m k v)"
  using storeClosedWf1 storeClosedWf2 storeClosedWf3 storeClosedWfDef wfMapV
  assms
  by (smt (verit) One_nat_def add_diff_cancel_left' tmap_lvl.simps(3,4)
      diff_diff_cancel diff_is_0_eq map_level_gt_0 mapval_ty_eq_ty321 plus_1_eq_Suc
      storeImpl.simps storePreserveTy type_of_val.simps(1,2) wf.cases wf_impl_wf_ty
      zero_neq_one)

```

2.13 Defining well formed type

set for well formed inner map values

```

definition wf_map_set :: "'a::absval val321 set" where
  "wf_map_set = {m. wf (MapV m)}"

```

```

lemma wf_map_bij: "wf v ↔ (∃ v'. v = LitV v') ∨ (∃ v'. v = AbsV v')
  ∨ (∃ m'. v = MapV m' ∧ m' ∈ wf_map_set)"
  apply (case_tac v)
  apply (simp add: wfLitV)
  apply (simp add: wfAbsV)

```

```

by (simp add: wf_map_set_def)

typedef for well formed inner map values

typedef (overloaded) 'a::absval wf_maps = "wf_map_set :: 'a::absval val321
set"
proof
  show "(Inr (Inr mAdd1)) ∈ wf_map_set"
    unfolding wf_map_set_def using wf_vAdd1 by simp
qed

wf maps are mapval

instantiation wf_maps :: (type) mapval begin
  fun mapval_ty_wf_maps where "mapval_ty_wf_maps x = mapval_ty (Rep_wf_maps
x)"
  instance .. end

lemma "mapval_ty (Abs_wf_maps (Inr (Inr mAdd1))) = (TT, TT)"
  by (simp add: Abs_wf_maps_inverse wf_map_set_def wf_vAdd1)

type for well formed values

type_synonym 'a wf_val = "('a, 'a wf_maps) val"

lift selectImpl and storeImpl

setup_lifting type_definition_wf_maps

lift_definition wf_select :: "'a::absval wf_val ⇒ 'a wf_val ⇒ 'a wf_val"
  is selectImpl
  using selectClosedWf wf_map_bij
  by (metis top1I val.exhaust
    val.pred_inject(2)[of top "λuu. uu ∈ wf_map_set"]
    val.pred_inject(3)[of top "λuu. uu ∈ wf_map_set"]
    val.pred_rel[of top "λuu. uu ∈ wf_map_set" "LitV _"]
    val.rel_inject(1)[of "eq_onp top"
      "eq_onp (λuu. uu ∈ wf_map_set)"])

lift_definition wf_store :: "'a::absval wf_val ⇒ 'a wf_val ⇒ 'a wf_val ⇒
'a wf_val"
  is storeImpl
  using storeClosedWf wf_map_bij
  by (metis top1I val.exhaust
    val.pred_inject(2)[of top "λuu. uu ∈ wf_map_set"]
    val.pred_inject(3)[of top "λuu. uu ∈ wf_map_set"]
    val.pred_rel[of top "λuu. uu ∈ wf_map_set" "LitV _"]
    val.rel_inject(1)[of "eq_onp top"
      "eq_onp (λuu. uu ∈ wf_map_set)"])

lift_definition wf_wf :: "'a::absval wf_val ⇒ bool"
  is wf .

```

```
lift_definition type_of_wf_val :: "'a::absval wf_val  $\Rightarrow$  ty"
  is type_of_val .
```

2.14 Lemmas hold for the new select and store

```
lemma wf_wf_val: "wf_wf x"
  apply (cases x)
  apply (simp add: wfLitV wf_wf.rep_eq)
  apply (simp add: wfAbsV wf_wf.rep_eq)
  by (simp add: Rep_wf_maps wf_map_bij wf_wf.rep_eq)
```

```
lemma type_of_val_transfer [transfer_rule]:
  "rel_fun (rel_val (=) cr_wf_maps) (=) type_of_val type_of_val"
  unfolding rel_fun_def cr_wf_maps_def
  apply (rule, rename_tac v_raw, rule, rename_tac v_lift)
  by (case_tac v_raw; case_tac v_lift; auto)
```

```
lemma Ax1:
  fixes m::"'a::absval wf_val"
  assumes "type_of_val m = TMap (type_of_val k) (type_of_val v)"
  shows "wf_select (wf_store m k v) k = v"
  using assms
  apply transfer
  using ArrayAxUpdate wf_wf_val
  by (metis eq_onp_top_eq_eq val.pred_rel wf_wf.abs_eq)
```

```
lemma Ex:
  assumes "m = MapV m'  $\wedge$  n = MapV n'"
  assumes "type_of_val m = type_of_val n"
  assumes " $\forall k. \text{type\_of\_val } k = \text{key\_ty } (\text{type\_of\_val } m) \longrightarrow \text{wf\_select } m \ k =$ 
wf_select n k"
  shows "m = n"
  using assms
  apply transfer
  apply auto[1]
  using extensionalityMapV wf_wf_val apply auto
  by (smt (verit) extensionalityMapV key_ty.simps(1) selectImpl.simps top1I
type_of_val.simps(3)
  val.pred_inject(1,2,3) wf_map_bij)
```

```

lemma Ax2_wf_val:
  shows "x = y  $\vee$  wf_select (wf_store m x v) y = wf_select m y"
  by (smt (verit, del_insts) ArrayAxStable Rep_wf_maps_inject id_apply
      map_fun_apply val.inj_map_strong wf_select_def wf_store.rep_eq)

```

2.15 Proof for VC Phase

```

fun key_tyC where "key_tyC (TMapC tk _) = tk" | "key_tyC _ = undefined"
fun val_tyC where "val_tyC (TMapC _ tv) = tv" | "val_tyC _ = undefined"
lemma key_tyC_preserved: " $\forall$  tk tv. key_tyC (TMapC tk tv) = tk" by simp
lemma val_tyC_preserved: " $\forall$  tk tv. val_tyC (TMapC tk tv) = tv" by simp

```

```

lemma map_type_safe_wf:
  shows "type_of_val (wf_select (MapV m) k) = val_ty (type_of_val (MapV m))"
  apply transfer
  using wf.simps wf_map_set_def by fastforce

```

```

lemma map_select_type_safe: " $\forall$  m k.
  let tk = vc_type_of_val k; tv = val_tyC (vc_type_of_val m)
  in vc_type_of_val m = TMapC tk tv  $\wedge$  vc_type_of_val k = tk  $\longrightarrow$ 
    vc_type_of_val (wf_select m k) = tv"
  apply (rule, case_tac m; simp) using map_type_safe_wf
  by (metis mapval_ty_wf_maps.elims type_of_val.simps(3) val_ty.simps(1))

```

```

lemma map_store_type_safe: " $\forall$  m k v.
  let tk = vc_type_of_val k; tv = vc_type_of_val v
  in (vc_type_of_val m = TMapC tk tv  $\wedge$ 
    vc_type_of_val k = tk)  $\wedge$ 
    vc_type_of_val v = tv  $\longrightarrow$ 
    vc_type_of_val (wf_store m k v) =
      TMapC tk tv"
  apply (rule, case_tac m; simp)
  apply transfer
  using storePreserveTy
  by (metis (no_types, lifting) ty_to_closed.simps(3) type_of_val.simps(3))

```

```

lemma type_of_mapval_closed:
  assumes "closed (type_of_val m)" "closed (type_of_val k)" "closed (type_of_val v)"
  shows "type_of_val m = TMap (type_of_val k) (type_of_val v)"
  by (metis assms(1,2,3,4) closed_inv2 closed_to_ty.simps(3) vc_type_of_val.simps)

```

```

lemma map_update:
  assumes " $\bigwedge$  v::('a::absval, 'a wf_maps) val. closed (type_of_val v)"
  shows " $\forall$  (m::('a, 'a wf_maps) val) k v.
    let tk = vc_type_of_val k; tv = vc_type_of_val v
    in vc_type_of_val m = TMapC tk tv  $\longrightarrow$  wf_select (wf_store m k v) k
    = v"

```

```
by (meson Ax1 assms type_of_mapval_closed)
```

2.16 Proof for Locale Assumptions

```
lemma locale_select: " $\bigwedge m k tk tv. \llbracket \text{type\_of\_val } m = \text{TMap } tk \text{ } tv; \text{type\_of\_val } k = tk \rrbracket$   

 $\implies \text{type\_of\_val } (wf\_select \ m \ k) = tv$ "  

by (metis map_type_safe_wf ty.simps(14,16) type_of_val.elims val_ty.simps(1))
```

```
lemma locale_store: " $\bigwedge m k v tk tv. \llbracket \text{type\_of\_val } m = \text{TMap } tk \text{ } tv; \text{type\_of\_val } k = tk; \text{type\_of\_val } v = tv \rrbracket$   

 $\implies \text{type\_of\_val } (wf\_store \ m \ k \ v) = \text{TMap } tk \text{ } tv$ "  

apply transfer  

using storePreserveTy by metis
```

```
lemma intintmap:  

assumes "type_of_val m = TMap TT TT"  

shows " $\exists j. (wf\_select \ m \ (\text{IntV } i)) = \text{IntV } j$ "  

by (simp add: assms locale_select tint_intv)
```

```
lemma inteq:  

assumes " $\exists i. x = \text{IntV } i$ " " $\exists j. y = \text{IntV } j$ "  

shows " $(x = y) = (\text{convert\_val\_to\_int } x = \text{convert\_val\_to\_int } y)$ "  

using assms(1,2) by force
```

```
lemma int_inverse_0: "type_of_val k = TT  $\implies (k = \text{IntV } i) = (\text{convert\_val\_to\_int } k = i)$ "  

using int_inverse_3 by auto
```

```
lemmas map_helper =  

  locale_select locale_store intintmap inteq  

  int_inverse_3 int_inverse_2 int_inverse_1 int_inverse_0 convert_val_to_int.simps
```

```
end
```